

Question Index - All 40 Questions

Q#	Question
Q1	Tekton basics - cloud-native CI/CD, Tasks, Pipelines, CRDs
Q2	Tekton architecture - PipelineRun, TaskRun, Steps, Workspaces
Q3	Tekton vs Jenkins - Kubernetes-native vs traditional CI
Q4	CircleCI basics - config.yml, jobs, workflows, executors
Q5	CircleCI features - orbs, caching, parallelism, contexts
Q6	Bamboo basics - Atlassian CI/CD, plans, stages, jobs, tasks
Q7	Bamboo integration - Jira, Bitbucket, deployment projects
Q8	CloudFormation basics - stacks, templates, resources, parameters
Q9	CloudFormation template anatomy - AWSTemplateFormatVersion, Resources, Outputs
Q10	CloudFormation vs Terraform - when to use which
Q11	Harness basics - intelligent CI/CD, pipelines, delegates, connectors
Q12	CI/CD platform comparison - Jenkins vs GitHub Actions vs GitLab vs CircleCI vs Tekton
Q13	Tekton Triggers - EventListener, TriggerBinding, TriggerTemplate
Q14	Tekton Catalog - reusable Tasks, community hub, versioning
Q15	Tekton Chains - supply chain security, signing, provenance
Q16	Tekton with ArgoCD - CI with Tekton, CD with ArgoCD pattern
Q17	CircleCI orbs - reusable packages, creating custom orbs
Q18	CircleCI advanced - matrix jobs, approval gates, scheduled pipelines
Q19	CircleCI vs GitHub Actions - comparison, migration patterns
Q20	Bamboo deployment projects - environments, release management
Q21	Bamboo specs - pipeline as code in Java/YAML
Q22	CloudFormation intrinsic functions - Ref, Fn::Sub, Fn::Join, Fn::If

Q23	CloudFormation nested stacks - modular templates, cross-stack references
Q24	CloudFormation change sets - preview changes before applying
Q25	CloudFormation drift detection - detecting manual changes
Q26	CloudFormation StackSets - multi-account, multi-region deployment
Q27	CloudFormation custom resources - Lambda-backed extensions
Q28	Harness pipelines - stages, steps, service/environment/infrastructure
Q29	Harness continuous verification - ML-based deployment validation
Q30	Harness feature flags - progressive delivery, targeting rules
Q31	Tekton at scale - multi-tenant, resource limits, pipeline governance
Q32	CloudFormation at scale - landing zones, Control Tower, Service Catalog
Q33	CI/CD platform selection framework - evaluating for enterprise
Q34	Multi-CI strategy - different tools for different teams
Q35	Tekton PipelineRun stuck pending - workspace PVC not bound
Q36	CircleCI build slow - caching, parallelism optimization
Q37	CloudFormation stack stuck in ROLLBACK_COMPLETE
Q38	CloudFormation update fails - resource replacement unexpected
Q39	Bamboo to GitHub Actions migration - strategy and pitfalls
Q40	CI/CD tool evaluation - choosing between 5 platforms for your org

SECTION 1 - Junior Core Concepts (Q1–Q12)

Q1 Tekton basics - cloud-native CI/CD

The Simple Answer

Tekton is a Kubernetes-native CI/CD framework. Unlike Jenkins (runs on a server, manages its own agents), Tekton runs entirely as Kubernetes Custom Resources. Pipelines, tasks, and runs are Kubernetes objects - managed with kubectl, stored in etcd, executed as Pods. If you know Kubernetes, you know how to operate Tekton.

The Analogy - Jenkins is a Restaurant, Tekton is a Food Truck Fleet

Jenkins is a traditional restaurant with a permanent kitchen (master server), hired staff (agents), and a fixed menu (installed plugins). Tekton is a fleet of food trucks (Kubernetes Pods) - each truck is spun up for a specific order (pipeline run), cooks the meal (executes steps), and disappears. No permanent kitchen. No idle staff. Scale by adding more trucks.

Key Concepts

Step - A single container that runs a command. The smallest unit. Like one instruction in a recipe. Task - A sequence of Steps that run in a single Pod. Steps share a workspace (filesystem). A Task is like a complete recipe - all steps run on the same kitchen counter. Pipeline - A sequence of Tasks. Each Task runs as a separate Pod. Tasks can run in parallel or sequentially. A Pipeline is a full meal - multiple recipes coordinated together. PipelineRun / TaskRun - An execution instance. Like pressing "start" - creates Pods, runs the pipeline, stores results. Each run is immutable - you can inspect logs, results, and artifacts after completion.

Q2 Tekton architecture - PipelineRun, Workspaces

The Simple Answer

PipelineRun - Triggers a Pipeline execution. Specifies: which Pipeline, parameter values, workspace bindings (where to store files), and service account (permissions). Created by: kubectl create, Tekton Triggers (webhook-driven), or Tekton Dashboard.

TaskRun - Executes a single Task. Created automatically by PipelineRun for each Task in the Pipeline. Or created standalone for individual Task execution.

Workspaces - Shared storage between Steps within a Task (emptyDir or PVC) and between Tasks in a Pipeline (PVC). The source code is cloned into a workspace. Build artifacts are written to a workspace. Subsequent Tasks read from the same workspace. PersistentVolumeClaims provide durable storage. emptyDir is ephemeral (lost after the Pod terminates).

Results - Tasks produce results (key-value strings) that subsequent Tasks can consume. Task A outputs the image digest. Task B uses that digest for deployment. This enables data passing between Pipeline stages.

Q3 Tekton vs Jenkins

The Simple Answer

Feature	Jenkins	Tekton
Architecture	Master/agent server	Kubernetes-native CRDs
Runs on	Any server (VM, container)	Kubernetes only
Scaling	Add agent nodes manually	Kubernetes auto-scales Pods
Configuration	Groovy/Declarative Jenkinsfile	Kubernetes YAML manifests
State	Persistent (master stores history)	Kubernetes-native (etcd)
Plugins	1800+ plugins (fragile ecosystem)	Tekton Catalog + any container
Resource usage	Always-on master + idle agents	Pods created on demand, zero idle
Learning curve	Lower (familiar to most DevOps)	Higher (requires K8s knowledge)
Best for	Existing teams, complex workflows	K8s-native teams, cloud-native orgs

****The honest assessment:**** Jenkins has the largest ecosystem and most flexibility. Tekton is cleaner and more Kubernetes-native but less mature. For Kubernetes-first organizations building new pipelines, Tekton is the natural choice. For existing Jenkins shops with complex pipelines, migration to Tekton is significant effort - consider GitHub Actions or GitLab CI instead.

Q4 CircleCI basics - config.yml, jobs, workflows

The Simple Answer

CircleCI is a cloud-native CI/CD platform. Configuration lives in `.circleci/config.yml` in your repository. CI runs in CircleCI's cloud (or self-hosted runners). Key strengths: fast execution (powerful cloud machines), excellent caching, and Docker-first approach.

Key Concepts

Jobs - A collection of steps that run in a single executor (Docker container, VM, or macOS). **Steps** - Individual commands: checkout (clone repo), run (execute command), save_cache, restore_cache, persist_to_workspace. **Workflows** - Orchestrate jobs: sequential, parallel, conditional, approval gates. **Executors** - The environment where jobs run. docker (most common - specify any Docker image), machine (full VM for Docker-in-Docker), macos (for iOS builds).

Example Config

```
version: 2.1. jobs: build: docker: [{image: node:18}]. steps: [checkout, run: npm install, run: npm test]. workflows: main: jobs: [build].
```

Q5 CircleCI features - orbs, caching, parallelism

Orbs

Reusable packages of CircleCI configuration. Like Helm charts for CI/CD. circleci/aws-ecr orb provides: build-and-push-image job (build Docker image, push to ECR in one step). circleci/kubernetes orb provides: kubectl commands. Use: orbs: aws-ecr: circleci/aws-ecr@8.2.1. Thousands available in the CircleCI registry. Create custom orbs for internal reuse.

Caching

save_cache saves directories between builds (node_modules, .m2, pip cache). restore_cache retrieves them. Key-based: checksum("package-lock.json") - cache invalidates when dependencies change. Dramatically speeds up builds - npm install with cached node_modules takes 5 seconds instead of 60.

Parallelism

parallelism: 4 runs 4 copies of a job simultaneously. CircleCI auto-splits test files across copies. A 20-minute test suite on 4 parallel containers finishes in 5 minutes. Tests must be independent (no shared state).

Q6 Bamboo basics - Atlassian CI/CD

The Simple Answer

Bamboo is Atlassian's CI/CD server. Tightly integrated with Jira (link builds to issues), Bitbucket (automatic build triggers), and Confluence (deployment documentation). Available as: Server (self-hosted, being deprecated) and Cloud (Atlassian cloud, limited features). Atlassian is migrating CI/CD to Bitbucket Pipelines - Bamboo Server reaches end of support in 2027.

Key Concepts

Plan - A build configuration (equivalent to a Jenkins job). Contains stages, jobs, and tasks. **Stage** - A group of jobs that run in parallel. Stages run sequentially. **Job** - Runs on an agent. Contains tasks. Jobs within a stage run in parallel. **Task** - A single action: source checkout, Maven build, script execution, artifact publishing. **Agent** - A build machine (local or remote). Bamboo assigns jobs to agents based on capabilities.

Q7 Bamboo integration - Jira, Bitbucket

Jira Integration

Bamboo links builds to Jira issues automatically. Commit messages containing JIRA-123 associate the build with that issue. Jira shows: which builds include changes for this issue, deployment status (which environment has this fix), and build logs. This traceability is Bamboo's strongest feature - auditors can trace from requirement (Jira) to code (Bitbucket) to build (Bamboo) to deployment.

Bitbucket Integration

Automatic build triggers on push, PR creation, and branch creation. Build status displayed on PRs (pass/fail). Branch-based build plans - each feature branch gets its own build. Merge checks - require successful build before merging.

Q8 CloudFormation basics - stacks, templates

The Simple Answer

AWS CloudFormation is AWS's native Infrastructure as Code service. You write a JSON or YAML template describing AWS resources, and CloudFormation creates, updates, and deletes them as a managed stack. It is to AWS what Terraform is to multi-cloud - but AWS-only and deeply integrated with AWS services.

The Analogy - An Architect's Blueprint

A CloudFormation template is an architect's blueprint. The blueprint describes every room (resource), door (connection), and electrical outlet (configuration). CloudFormation is the construction company - it reads the blueprint and builds the house. If you modify the blueprint, CloudFormation figures out what to change (add a room, remove a wall) without rebuilding the entire house.

Key Concepts

Template - YAML or JSON file describing resources. Stack - An instance of a template. A running set of AWS resources managed as a unit. Create, update, or delete the stack - all resources move together. Resources - The AWS resources to create: EC2 instances, VPCs, S3 buckets, RDS databases, Lambda functions. Parameters - Input values at stack creation: instance type, environment name, VPC CIDR. Outputs - Values exported from the stack: VPC ID, load balancer DNS name, database endpoint. Used for cross-stack references.

Q9 CloudFormation template anatomy

The Simple Answer

A template has these sections (in order): AWSTemplateFormatVersion: "2010-09-09" (always this value). Description: Human-readable description. Parameters: Input values with types, defaults, and constraints. Mappings: Static lookup tables (AMI IDs per region). Conditions: Conditional resource creation (create this resource only in production). Resources: (required) The AWS resources to create. Each resource has a Type (AWS::EC2::Instance) and Properties (InstanceType, ImageId, etc.). Outputs: Values to export (LoadBalancerDNS, VPCId).

Resources Section - The Core

Resources: WebServer: Type: AWS::EC2::Instance. Properties: InstanceType: !Ref InstanceTypeParam. ImageId: ami-12345678. SecurityGroupIds: [!Ref WebSG]. Tags: [{Key: Name, Value: web-server}]. The Type defines what AWS resource to create. Properties configure it. !Ref references other resources or parameters within the template.

Q10 CloudFormation vs Terraform

The Simple Answer

Feature	CloudFormation	Terraform
Provider	AWS only	Multi-cloud (AWS, GCP, Azure, 1000+ providers)
Language	JSON or YAML	HCL
State	Managed by AWS (automatic)	State file (you manage - S3 backend)
Drift detection	Built-in	terraform plan (partial)
Rollback	Automatic on failure	Manual
Cost	Free (pay for resources only)	Free OSS, paid Terraform Cloud
Modules	Nested stacks, Modules	Modules (more mature)
Preview	Change Sets	terraform plan
Speed	Slower (AWS API-dependent)	Faster (parallel by default)
Learning curve	Lower for AWS-only teams	Lower for multi-cloud teams

When to Use CloudFormation

All-in on AWS with no multi-cloud plans. Want automatic rollback on failure (CloudFormation rolls back the entire stack). Need deep AWS integration (Service Catalog, Control Tower, Landing Zones). Want AWS-managed state (no S3 backend to configure).

When to Use Terraform

Multi-cloud or hybrid. Need to manage non-AWS resources (GitHub repos, Datadog monitors, PagerDuty services). Prefer HCL over YAML/JSON. Want a larger ecosystem of providers and modules. Team already knows Terraform.

****The pragmatic answer:**** Most organizations use Terraform even for AWS-only infrastructure because: HCL is more expressive than CloudFormation YAML, the module ecosystem is larger, and

Terraform skills are transferable across clouds. CloudFormation wins when you need: AWS-managed state, automatic rollback, or integration with AWS Control Tower/Service Catalog.

Q11 Harness basics - intelligent CI/CD

The Simple Answer

Harness is a modern CI/CD platform that emphasizes intelligent automation - ML-based deployment verification, automatic rollback, and cost optimization. Founded by the former VP of DevOps at AppDynamics, Harness focuses on reducing deployment risk through AI/ML.

Key Concepts

Pipelines - Multi-stage workflows (build, test, deploy, verify). Visual pipeline builder. Delegates - Agents installed in your infrastructure (Kubernetes, VMs). Connect Harness to your environment without exposing your infrastructure to the internet. Connectors - Integrations with: cloud providers, Git repos, artifact registries, monitoring tools. Service - What you deploy (Docker image, Helm chart, Lambda function). Environment - Where you deploy (dev, staging, prod Kubernetes cluster). Infrastructure Definition - How to deploy (which cluster, namespace, service account).

What Makes Harness Different

Continuous Verification (CV) - After deployment, Harness queries your monitoring tools (Prometheus, Datadog, New Relic) and uses ML to detect anomalies. If error rates increase or latency degrades, Harness automatically rolls back - no human intervention needed. Cost Management - Tracks cloud spend per deployment, per service, per team. Identifies cost anomalies. Feature Flags - Built-in feature flag management. Progressive delivery with targeting rules.

Q12 CI/CD platform comparison

The Simple Answer

Feature	Jenkins	GitHub Actions	GitLab CI	CircleCI	Tekton
Hosting	Self-hosted	Cloud (GitHub)	Cloud or self-hosted	Cloud or self-hosted	Kubernetes
Config	Groovy/Jenkinsfile	YAML workflows	.gitlab-ci.yml	.circleci/config.yml	K8s YAML CRDs
Marketplace	1800 plugins	15000+ Actions	Templates	Orbs	Tekton Catalog
Strength	Flexibility, maturity	GitHub integration	All-in-one platform	Speed, caching	K8s-native

Weakness	Maintenance burden	GitHub lock-in	GitLab lock-in	Cost at scale	K8s required
Best for	Complex, legacy	GitHub-hosted repos	GitLab-hosted repos	Fast builds, Docker	K8s-native orgs

SECTION 2 - Mid-Level Production Patterns (Q13–Q30)

Q13 Tekton Triggers - EventListener, webhooks

The Simple Answer

Tekton Triggers automatically creates PipelineRuns in response to external events (Git webhooks, Slack commands, monitoring alerts).

EventListener - A Kubernetes Service that listens for incoming webhooks. Receives POST requests from GitHub, GitLab, or any HTTP source. TriggerBinding - Extracts data from the webhook payload: repository URL, branch name, commit SHA. TriggerTemplate - Defines the PipelineRun to create, using the extracted data as parameters.

The Flow

GitHub push → webhook POST to EventListener → TriggerBinding extracts repo, branch, SHA → TriggerTemplate creates a PipelineRun with those parameters → Pipeline executes (clone, build, test, deploy). Fully automated - no manual trigger needed.

Q14 Tekton Catalog - reusable Tasks

The Simple Answer

The Tekton Catalog (hub.tekton.dev) is a community repository of reusable Tasks. git-clone (clone a repository), buildah (build container images), kubectl-deploy (deploy to Kubernetes), kaniko (build images in Kubernetes without Docker daemon), helm-upgrade (deploy Helm charts), and hundreds more. Install: `kubectl apply -f https://hub.tekton.dev/tekton/task/git-clone/0.9`. Reference in your Pipeline: `taskRef: name: git-clone`.

Q15 Tekton Chains - supply chain security

The Simple Answer

Tekton Chains automatically signs TaskRun and PipelineRun results with cryptographic attestations. After a build completes, Chains generates a SLSA provenance attestation: what was built, from which source, by which pipeline, on which cluster. The attestation is signed (cosign/sigstore) and stored alongside the artifact. Kubernetes admission webhooks can verify: "only deploy images with valid Tekton Chains attestations" - ensuring every production image was built by your CI pipeline, not manually.

Q16 Tekton with ArgoCD - CI + CD pattern

The Simple Answer

Tekton handles CI (build, test, push image). ArgoCD handles CD (deploy to Kubernetes from Git). The pattern: Tekton Pipeline clones code → builds image → pushes to registry → updates the image tag in the GitOps config repo → ArgoCD detects the Git change → syncs the new image to the cluster.

Why Separate CI and CD

CI (Tekton) needs: build tools, test frameworks, registry access. Runs on every commit. CD (ArgoCD) needs: cluster access, GitOps reconciliation. Runs on Git config changes. Different security boundaries - CI has push access to the registry. CD has deploy access to the cluster. Neither needs both.

Q17 CircleCI orbs - reusable packages

The Simple Answer

Orbs are reusable packages of CircleCI configuration - commands, jobs, and executors. Like Ansible roles or Helm charts for CI/CD. Published to the CircleCI registry with versioning. Use community orbs for common tasks: circleci/aws-ecr, circleci/kubernetes, circleci/slack. Create private orbs for internal patterns.

Creating Custom Orbs

Define reusable commands, jobs, and executors in a YAML structure. Publish: circleci orb publish my-org/deploy@1.0.0. Teams use it: orbs: deploy: my-org/deploy@1.0.0. Update the orb once - all consuming pipelines get the update (if using non-pinned or minor-version ranges).

Q18 CircleCI advanced - matrix, approvals

Matrix Jobs

Test across multiple dimensions: matrix: parameters: node-version: ["16", "18", "20"]; os: [ubuntu, alpine]. Generates 6 job combinations automatically. Each runs in parallel.

Approval Gates

workflows: deploy: jobs: [build, test, hold: {type: approval, requires: [test]}, deploy-prod: {requires: [hold]}]. The pipeline pauses at "hold" until a human approves in the CircleCI UI. Used for production deployments requiring manual approval.

Scheduled Pipelines

Run pipelines on a schedule (cron syntax). Used for: nightly builds, weekly dependency updates, scheduled performance tests.

Q19 CircleCI vs GitHub Actions

The Simple Answer

Feature	CircleCI	GitHub Actions
Config location	.circleci/config.yml	.github/workflows/*.yml
Reuse mechanism	Orbs	Actions (marketplace)
Marketplace	Smaller (orbs)	Larger (15000+ actions)

Caching	Excellent (explicit, key-based)	Good (actions/cache)
Docker support	First-class (docker executor)	Good (docker commands)
Self-hosted runners	Yes	Yes
Pricing	Per-credit (compute time)	Free for public repos, per-minute for private
Best for	Docker-heavy, fast builds	GitHub-hosted projects

****Migration tip:**** CircleCI jobs map to GitHub Actions jobs. CircleCI orbs map to GitHub Actions marketplace actions. CircleCI contexts map to GitHub environments with secrets. Most migrations are straightforward - the YAML syntax differs but the concepts are the same.

Q20 Bamboo deployment projects

The Simple Answer

Bamboo separates build (Plans) from deployment (Deployment Projects). A Deployment Project defines: which build plan provides the artifact, which environments to deploy to (dev, staging, prod), and deployment tasks per environment. Each environment can have: manual triggers (production requires approval), automatic triggers (deploy to dev on every successful build), and permissions (only the release team can deploy to production).

Q21 Bamboo specs - pipeline as code

The Simple Answer

Bamboo Specs allows defining build plans in Java or YAML (stored in the repository). When pushed to Bitbucket, Bamboo automatically creates or updates the plan. YAML syntax: stages, jobs, tasks, triggers, and variables. Java syntax: `Plan plan = new Plan(project, "Build", "BUILD").stages(new Stage("Build").jobs(job))`. Java specs are more powerful but YAML is simpler for most use cases.

Q22 CloudFormation intrinsic functions

The Simple Answer

Intrinsic functions add logic to CloudFormation templates. `!Ref` - Reference a parameter or resource: `!Ref VPCId` returns the VPC ID. `Fn::Sub` (or `!Sub`) - String substitution: `!Sub "arn:aws:s3:::${BucketName}/*"`. `Fn::Join` - Join strings: `!Join ["-", [!Ref Project, !Ref Environment, "bucket"]]`. `Fn::Select` - Select from a list: `!Select [0, !GetAZs "us-east-1"]`. `Fn::If` - Conditional value: `!If [IsProduction, "m5.xlarge", "t3.medium"]`. Requires a Condition defined in the Conditions section.

`Fn::GetAtt` - Get a resource attribute: `!GetAtt WebServer.PublicIp`. `Fn::ImportValue` - Import an output from another stack (cross-stack references).

Q23 CloudFormation nested stacks

The Simple Answer

Nested stacks modularize large templates. A parent stack includes child stacks as resources (Type: `AWS::CloudFormation::Stack`). The child stack is a separate template file stored in S3. Parent passes parameters to children. Children export outputs consumed by the parent or siblings.

Pattern

Parent template: VPC stack (networking) → Security stack (security groups, IAM) → Application stack (EC2, ALB, RDS). Each is a separate template, independently updatable. Changes to the security stack do not require redeploying the VPC stack.

Cross-Stack References

Alternative to nested stacks: independent stacks with exported outputs. Stack A exports `VPCId`: Outputs: `VPCId`: Value: `!Ref VPC`, Export: Name: `prod-vpc-id`. Stack B imports it: `!ImportValue prod-vpc-id`. Looser coupling than nested stacks - each stack is fully independent.

Q24 CloudFormation change sets

The Simple Answer

Change sets preview what CloudFormation will change BEFORE applying the update. Like terraform plan for CloudFormation. Create a change set: `aws cloudformation create-change-set --stack-name my-stack --template-body file://template.yaml`. Review: `aws cloudformation describe-change-set` shows: which resources will be added, modified, or deleted, and whether modifications require replacement (downtime). Execute only if the changes are expected: `aws cloudformation execute-change-set`.

****Always use change sets in production.**** Direct stack updates (`aws cloudformation update-stack`) apply immediately with no preview. A typo in the template can delete your production database. Change sets prevent surprises.

Q25 CloudFormation drift detection

The Simple Answer

Drift occurs when the actual AWS resource configuration differs from the CloudFormation template - someone modified a resource manually (console click, CLI command). CloudFormation drift detection compares: what the template says vs what actually exists. `aws cloudformation detect-stack-drift --stack-name my-stack`. Results show: which resources drifted, which properties changed, and the expected vs actual values. Remediate by: updating the template to match reality, or updating the resource to match the template (stack update).

Q26 CloudFormation StackSets - multi-account, multi-region

The Simple Answer

StackSets deploy a single CloudFormation template across multiple AWS accounts and regions simultaneously. Use case: deploy security baselines (GuardDuty, Config Rules, CloudTrail) to all 50 accounts in your AWS Organization. One StackSet → 50 accounts × 4 regions = 200 stacks created automatically. New accounts added to the Organization automatically receive the StackSet.

Q27 CloudFormation custom resources - Lambda-backed

The Simple Answer

When CloudFormation does not natively support a resource or operation, custom resources extend it. A Lambda function handles the create, update, and delete lifecycle events. Use cases: create resources in non-AWS systems (GitHub repos, Datadog monitors), run custom validation before creating resources, look up dynamic values (latest AMI ID), or perform operations CloudFormation does not support natively.

How It Works

Template defines: `Type: Custom::MyResource`, `ServiceToken: !GetAtt MyLambda.Arn`, `Properties:` (custom parameters). CloudFormation invokes the Lambda with a Create/Update/Delete event. The Lambda performs the operation and sends a success/failure response to a pre-signed S3 URL. CloudFormation proceeds based on the response.

Q28 Harness pipelines - stages, steps

The Simple Answer

Harness pipelines have a visual builder with a code-behind YAML. Stages: Build (CI), Deploy (CD), Feature Flag, Custom. Steps within Deploy stage: Fetch artifact → Deploy to infrastructure (K8s apply, Helm, ECS, Lambda) → Verify (continuous verification) → Rollback on failure. Approval steps: Jira approval, Harness UI approval, custom webhook approval. Infrastructure definitions: Which cluster, namespace, service account, and cloud provider to target.

Q29 Harness continuous verification

The Simple Answer

After deploying, Harness queries your monitoring tools (Prometheus, Datadog, New Relic, AppDynamics, Splunk) for metrics and logs. ML algorithms compare: pre-deployment baseline vs post-deployment behavior. If error rate increases, latency degrades, or log error patterns change, Harness classifies the deployment as unhealthy and automatically triggers rollback.

Why This Matters

Traditional canary deployments require manual metric review or hand-crafted alert rules. Harness automates this with ML - no custom Prometheus rules needed. The ML learns your normal patterns and

detects anomalies automatically. Claims: reduces MTTR by 90% through automatic detection and rollback.

Q30 Harness feature flags

The Simple Answer

Harness includes a built-in feature flag service. Define flags in the Harness platform. Target: specific users (beta testers), percentages (5% of traffic), attributes (users in region=EU), or environments (enabled in staging, disabled in production). SDKs available for: Java, Python, Node.js, Go, React, Android, iOS.

Integration with Pipelines

Deploy code with feature flag disabled → Enable for 5% of users → Monitor (continuous verification) → If healthy, increase to 25%, 50%, 100% → If unhealthy, disable flag (instant rollback without redeployment). This decouples deployment (code reaches production) from release (users see the feature).

SECTION 3 - Senior Architecture (Q31–Q34)

Q31 Tekton at scale - multi-tenant, governance

The Simple Answer

Running Tekton for 50+ teams requires: namespace isolation (each team's pipelines in their own namespace), resource quotas (prevent one team's pipeline from consuming all cluster resources), RBAC (teams can only run their own pipelines), and pipeline governance (approved Tasks only, no arbitrary container images in Steps).

Multi-Tenancy Pattern

Namespace per team. Each namespace has: ResourceQuota (CPU, memory limits for PipelineRuns), LimitRange (per-Pod limits), NetworkPolicy (pipeline Pods cannot access other namespaces), and RoleBinding (team can create PipelineRuns but not modify Tasks). Shared Tasks in a central namespace - teams reference but cannot modify.

Q32 CloudFormation at scale - Landing Zones, Control Tower

The Simple Answer

AWS Control Tower - Sets up a multi-account AWS environment with: organizational units, account factory (automated account creation), guardrails (preventive and detective), and a landing zone (baseline networking, logging, security). CloudFormation StackSets deploy guardrails across all accounts.

AWS Service Catalog - Curated catalog of approved CloudFormation templates. Developers can self-service launch approved infrastructure (VPCs, EKS clusters, RDS databases) without writing CloudFormation. IT controls what is available and with what parameters. Governance + developer velocity.

Q33 CI/CD platform selection framework

The Decision Framework

Team and skills: Do developers already know a platform? Jenkins skills are common. Tekton requires Kubernetes expertise. GitHub Actions requires GitHub. Infrastructure: Cloud-only? Kubernetes-native? On-premises? This narrows options immediately. Integration requirements: Jira integration → Bamboo or Harness. GitHub repos → GitHub Actions. GitLab repos → GitLab CI. Scale: 5 developers → any platform works. 500 developers → need: multi-tenancy, RBAC, audit logging, shared pipelines. Cost: Open source (Jenkins, Tekton) vs SaaS (CircleCI, Harness, GitHub Actions). SaaS reduces operational burden but costs per-minute/per-seat. Security requirements: Air-gapped environments → self-hosted only (Jenkins, Tekton, GitLab). Compliance needs → audit trails, signed artifacts (Tekton Chains, Harness).

Q34 Multi-CI strategy

The Simple Answer

Large organizations often use multiple CI/CD platforms: GitHub Actions for open-source and public projects, Jenkins for legacy pipelines, Tekton for Kubernetes-native workloads, and GitLab CI for specific teams. This is pragmatic - forcing all teams onto one platform is expensive and disruptive. The key: standardize the artifacts (Docker images, Helm charts) and deployment targets (Kubernetes, ArgoCD) even if the CI platform varies. The CD side (ArgoCD, Harness) does not care which CI built the image - it deploys from the registry.

SECTION 4 - Tricky Scenarios (Q35–Q40)

Q35 Tekton PipelineRun stuck pending

Step-by-Step

Step 1 - Check PipelineRun status. `kubectl describe pipelinerun my-run -n ci`. Look for: conditions, events, and pod status. Common message: "TaskRun pending: workspace PVC not bound."

Step 2 - Check the workspace PVC. `kubectl get pvc -n ci`. Is the PVC in Pending state? If so: the StorageClass cannot provision the volume (check StorageClass exists, quota not exceeded, cloud permissions).

Step 3 - Check the Pod. `kubectl get pods -n ci | grep my-run`. If no Pod exists, the scheduler cannot place it: insufficient resources (CPU/memory), node affinity/taints preventing scheduling, or resource quotas exceeded.

Step 4 - Fix. For PVC issues: verify StorageClass, increase quota, or use `emptyDir` for ephemeral workspaces. For resource issues: increase namespace quotas or clean up old PipelineRuns consuming resources.

Q36 CircleCI build slow - optimization

Step-by-Step

Step 1 - Enable timing. CircleCI shows time per step. Identify the slowest steps.

Step 2 - Add caching. Cache dependencies: `save_cache` with checksum key for `package-lock.json/requirements.txt`. Cache Docker layers: docker layer caching (DLC) feature. Cache build outputs between steps.

Step 3 - Parallelize tests. `parallelism: 4` splits tests across 4 containers. CircleCI's test splitting by timing data assigns tests to containers for optimal balance.

Step 4 - Use larger resource class. `resource_class: large` provides 4 CPU, 8 GB RAM (vs 2 CPU, 4 GB for medium). Costs more but compiles faster. Math: if large finishes in 5 minutes vs medium in 10 minutes, and you run 100 builds/day, large saves 500 minutes/day.

Step 5 - Docker layer caching. Enable DLC: `docker_layer_caching: true`. Docker builds reuse layers from previous builds - only changed layers are rebuilt. Reduces 10-minute Docker builds to 2 minutes.

Q37 CloudFormation stack stuck in ROLLBACK_COMPLETE

What Happened

A stack creation or update failed. CloudFormation attempted rollback but the rollback also failed. The stack is stuck in `ROLLBACK_COMPLETE` or `UPDATE_ROLLBACK_FAILED` state.

Step-by-Step

ROLLBACK_COMPLETE - The stack was being created, failed, and rolled back successfully. The stack exists but has no resources. You CANNOT update a stack in **ROLLBACK_COMPLETE** - you must delete and recreate it. `aws cloudformation delete-stack --stack-name my-stack`. Fix the template error. Create the stack again.

UPDATE_ROLLBACK_FAILED - An update failed and the rollback ALSO failed. The stack is in an inconsistent state. Fix: `aws cloudformation continue-update-rollback --stack-name my-stack` (skip the resources that cannot be rolled back). If that fails: manually fix the resources that are blocking rollback, then `continue-update-rollback`.

Common Causes

Resource creation failed - IAM permissions missing, resource limit exceeded, invalid property value.

Rollback failed - A resource created during the update cannot be deleted (S3 bucket not empty, ENI in use). Fix the blocking resource manually, then continue rollback.

Q38 CloudFormation update fails - unexpected replacement

What Happened

You updated a CloudFormation template to change a property. CloudFormation deleted the old resource and created a new one (replacement) instead of modifying in place. The database was deleted with all data.

Why It Happens

Some property changes require resource replacement - CloudFormation cannot modify the resource in place. For RDS: changing the engine (MySQL → PostgreSQL) requires replacement. For EC2: changing the instance type is an in-place modification, but changing the AMI requires replacement. CloudFormation documentation marks each property as "Update requires: No interruption," "Some interruption," or "Replacement."

Prevention

Always use change sets before updating. The change set shows: "RDS instance: REPLACEMENT" - a clear warning that the database will be deleted and recreated. Never update directly. For critical resources: add `DeletionPolicy: Retain` - CloudFormation will not delete the resource even during replacement. Enable RDS deletion protection. Take snapshots before any change set that shows replacement.

Q39 Bamboo to GitHub Actions migration

Step-by-Step

Step 1 - Inventory. List all Bamboo plans, their triggers, and deployment projects. Map each to a GitHub Actions workflow.

Step 2 - Map concepts. Bamboo Plan → GitHub Actions workflow. Bamboo Stage → Jobs with needs dependencies. Bamboo Job → Job. Bamboo Task → Step (run or action). Bamboo Variables → GitHub secrets and environment variables. Bamboo Agent capabilities → Runner labels (runs-on).

Step 3 - Migrate incrementally. Start with a simple, non-critical plan. Recreate as a GitHub Actions workflow. Run both in parallel for 2 weeks. Compare results. Once validated, disable the Bamboo plan.

Step 4 - Handle Jira integration. Bamboo's native Jira integration does not exist in GitHub Actions. Use: Jira GitHub Actions (atlassian/gajira) for: creating issues, updating issue status, adding build links. Or use Jira webhooks triggered by GitHub Actions.

Step 5 - Handle deployment projects. Bamboo deployment projects with environment approvals → GitHub Actions environments with required reviewers and protection rules.

Q40 CI/CD tool evaluation - choosing for your org

The Evaluation

Requirements gathering: Number of teams (10 teams), repository platform (GitHub), infrastructure (AWS EKS), security needs (SOC 2, artifact signing), budget (open to SaaS), existing tools (Jira, Slack).

Shortlist: GitHub Actions (GitHub-native), Tekton (K8s-native), Harness (intelligent CD), Jenkins (flexibility).

Evaluation criteria with weights: Developer experience (25%) - how easy is it for developers to create and debug pipelines? Integration (20%) - GitHub, Jira, Slack, EKS, ECR integration. Scale and governance (20%) - multi-team, RBAC, audit trails. Security (15%) - artifact signing, secret management, SLSA. Cost (10%) - per-seat, per-minute, or self-hosted costs. Operations (10%) - maintenance burden, upgrades, monitoring.

Decision: GitHub Actions for CI (deepest GitHub integration, 15000+ marketplace actions, low operational burden, developers already use GitHub daily). ArgoCD for CD (GitOps-native, multi-cluster, independent of CI platform). Rationale: CI is tightly coupled to the code platform (GitHub). CD is tightly coupled to the deployment platform (Kubernetes). Separate them for maximum flexibility.

Interview Tips

For Junior Roles

Focus on Q1-Q12. Know what Tekton, CircleCI, Bamboo, CloudFormation, and Harness ARE at a high level. Explain Tekton vs Jenkins. Know CloudFormation basics. Understand the CI/CD platform landscape.

For Mid-Level Roles

Q1-Q30. Be ready to discuss: Tekton Triggers and ArgoCD integration, CircleCI orbs and caching, CloudFormation intrinsic functions and change sets, and Harness continuous verification. Know how to optimize a slow CI pipeline.

For Senior Roles

All 40 questions. Platform selection framework, multi-CI strategy, CloudFormation at scale (StackSets, Control Tower), and Tekton governance separate senior from mid-level. Be ready to evaluate and recommend a CI/CD platform for an organization.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Tekton	K8s-native CI/CD. Tasks and Pipelines as CRDs. Pods on demand. Zero idle.
Tekton Triggers	EventListener + TriggerBinding + TriggerTemplate = automated PipelineRuns.
Tekton Chains	Supply chain security. Signs TaskRuns. SLSA provenance. Automatic.
CircleCI	Cloud CI. Fast. Excellent caching. Docker-first. Orbs for reuse.
CircleCI orbs	Reusable packages. Like Helm charts for CI. Community and private.
Bamboo	Atlassian CI/CD. Deep Jira/Bitbucket integration. End of support 2027.
CloudFormation	AWS-native IaC. JSON/YAML templates. Stacks. Automatic rollback.
CFN vs Terraform	CFN for AWS-only, managed state, auto-rollback. Terraform for multi-cloud.
Change sets	Preview before applying. Always use for production. Prevents data loss.
StackSets	Multi-account, multi-region deployment. Landing zones. Compliance baselines.
Drift detection	Finds manual changes. Compare template vs actual. Remediate to match.
Custom resources	Lambda-backed extensions. For resources CFN does not support natively.
Harness	ML-based continuous verification. Automatic rollback on anomaly detection.
Feature flags	Decouple deployment from release. Progressive delivery. Instant rollback.
Platform selection	Match to: team skills, infrastructure, integrations, scale, security, cost.
Multi-CI strategy	Different CI tools OK. Standardize artifacts and CD. ArgoCD does not care which CI built it.